

HMM Algorithms for Part-of-Speech Tagging

Ezgi Bas

bas.ez@northeastern.edu

Abstract

This project investigates the application of Hidden Markov Models (HMMs) for Part-of-Speech (PoS) tagging, comparing supervised learning approaches with unsupervised Expectation-Maximization (EM) training. I formally define the PoS tagging problem and provide detailed explanations of the Viterbi algorithm for decoding and the EM algorithm for parameter estimation. My experiments systematically vary the amount of training data for both supervised and EM-based approaches, as well as batch sizes for EM training. Performance is evaluated using accuracy, precision, recall, and F1 scores. Results demonstrate that supervised HMM training consistently outperforms the EM-based unsupervised approach. I observe that smaller batch sizes in EM training lead to convergence at smaller log probabilities. The paper concludes with a discussion on the limitations of the EM algorithm for PoS tagging and proposes potential improvements, including the implementation of factored HMMs and direct probability calculation instead of log probability manipulation.

Introduction

Part-of-Speech (PoS) tagging is a fundamental task in natural language processing that involves assigning grammatical categories (such as noun, verb, adjective) to words in a text. This labeling is essential for various downstream NLP applications, including syntactic parsing, information extraction, and machine translation. The task presents challenges due to word ambiguity, contextual dependencies, and the presence of unknown words in real-world texts (Chiche and Yitagesu 2022).

Hidden Markov Models (HMMs) have long been established as effective statistical models for sequence labeling tasks, including PoS tagging. These models capture both the emission probabilities (likelihood of a word given a tag) and transition probabilities (likelihood of moving from one tag to another) to determine the most likely sequence of tags for a given sentence.

In this project, I focus on two primary approaches to HMM-based PoS tagging: supervised learning and prediction with the Viterbi algorithm and unsupervised learning using the Expectation-Maximization (EM) algorithm combined with Viterbi decoding. While the supervised approach requires manually annotated training data, the EM algorithm

offers the potential advantage of learning from unannotated text. Our work provides a formal description of the PoS tagging problem and detailed explanations of both the Viterbi and EM algorithms. I conduct a series of experiments on the Brown corpus to systematically evaluate:

1. The impact of varying amounts of training data on supervised HMM performance
2. The effect of varying amounts of training data on EM-based HMM performance
3. The influence of batch size variations during EM training

The experimental results consistently show that supervised training outperforms EM-based approaches across all evaluation metrics. I observe that when varying batch sizes in EM training, smaller batches tend to converge at smaller log probabilities, suggesting potential optimization challenges.

The final section of this paper discusses the observed limitations of the EM algorithm for PoS tagging and proposes several potential improvements. In particular, I suggest implementing factored HMMs that can capture more complex dependencies and working directly with probabilities rather than log probabilities to avoid numerical underflow issues that may affect performance.

Background

Hidden Markov Models are a widely-used stochastic tool for various NLP tasks (Chiche and Yitagesu 2022). They are particularly useful in Part-of-Speech (PoS) tagging as their structure parallels the structure of the problem. In PoS tagging, we must find the underlying structure (the PoS) of a sentence. Using HMMs, we can represent the PoS tags as hidden states to be estimated, and the words in the sentence as emissions from these states. For a formal description, see “Project Description”.

Related Work

There are three main approaches to PoS tagging: rule-based, stochastic, and machine learning (Chiche and Yitagesu 2022).

Rule-based is the simplest to understand, however the hardest to implement well. In short, parts of speech are assigned based on certain rules about the word’s structure. For

example, if a word ends with “s”, we may assign it as a plural noun. However, this is an issue when we want to analyze a word like “speaks”, which is a verb. It is also difficult to distinguish tags for words that may be either part of speech given the context. For example “will” in “I will run.” compared to “I have free will”.

HMMs fall within the category of stochastic approaches. Another approach would be Naive Bayes, where one can simply count how many times each word appears as a certain PoS in the data and maximize expectation over a sentence. This may work well with a large, diverse set of data. However, the issue arises with the fact that it does not take into consideration the relationships words have with each other. For example, nouns are likely to follow determiners, but not adverbs in English. Naive Bayes classifiers operate on a word-by-word basis and don’t consider n-grams.

Convolutional Neural Networks, Feedforward Neural Networks, and Deep Learning approaches are also widely used in PoS tagging. The main reason I have chosen to use an HMM over a neural network is that I wanted to gain familiarity with a new algorithm and get programming experience with HMMs.

Project Description

This project consists of two main algorithms: the Viterbi algorithm and the Expectation-Maximization (EM) algorithm. An HMM model λ consists of:

1. A set of possible hidden states S
2. A set of possible emissions E
3. Initial probabilities π such that $\pi_i = P(X_1 = i)$ where $i \in S$
4. Transition probabilities a such that $a_{ij} = P(X_{t+1} = j | X_t = i)$ where $i, j \in S$
5. Emission probabilities b such that $b_{ik} = P(O_t = k | X_t = i)$ where $i \in S$ and $k \in E$

The sequence of hidden states is represented by $X_1, \dots, X_T$ where X_i can take any value in S . Similarly, the sequence of observed states $O_1, \dots, O_T$ where O_i can take any value in E .

Viterbi Algorithm

The Viterbi algorithm is concerned with finding the likeliest sequence of hidden states $X_1, \dots, X_T$ given a full HMM λ and a sequence of observations $O_1, \dots, O_T$.

The Viterbi algorithm keeps track of two variables:

1. $V_t(i)$, The probability of state i at time-step t , given $O_1, \dots, O_t$
2. $\psi_t(i)$ The most likely path leading to state i at time-step t , given $O_1, \dots, O_t$

Notice that $\psi_t(i)$ ensures that the entire *sequence* is maximized, as opposed to just the probability of one tag for one word. The algorithm consists of three phases: Initialization, Recursion, and Termination. These phases are illustrated with formal definitions and pseudocode below.

Initialization

$$V_1(i) = \pi_i b_i(O_1)$$

$$\psi_1(i) = \emptyset$$

```

V ← [{}]
ψ ← [{}]
T ← len(sequence)
for state ∈ S do
    V[1][state] ← π(state) * b1(state)
    ψ[1][state] ← {state}
end for

```

The initialization steps set the base cases for each variable. For V , the probability of i at time 1 given the observation at time 1 is simply the initial probability of i multiplied by the emission probability of the first observation from i . The path leading up to the first step is empty.

Recursion

$$V_t(j) = \max_{i \in S} [V_{t-1}(i) a_i(j)] b_j(O_t)$$

$$\psi_t(j) = \{\text{argmax}_{i \in S} [V_{t-1}(i) a_i(j)]\} + \{j\}$$

```

for t ∈ (1, ..., T] do
    emission ← sequence[t]
    for j ∈ S do
        max_prob ← -∞
        max_state ← ∅
        for i ∈ S do
            prob ← V[t-1][i] * ai * bj(emission)
            max_prob ← max(max_prob, prob)
            if prob = max_prob then max_state ← i
            end if
        end for
        V[t][j] ← max_prob
        ψ[t][j] ← ψ[t-1][max_state] + {j}
    end for
end for

```

The recursion step calculates all of the V and ψ values, using a dynamic programming approach. For the V values, we take the maximum probability of transitioning from the previous state i to the current state j , given the observations leading up to t . This requires multiplying $V(i)$ for the previous time-step with the transition probability from i to j , and the emission probability of the current observation. ψ is the argmax value of this calculation, i.e. the previous state that maximizes the probability of j being the current state.

Termination

$$\psi_T(\text{argmax}[V_T])$$

```

best_final_state ← argmax(V[T])
best_path ← ψ[T][best_final_state]
return best_path

```

The termination step extracts and returns the best path leading up to the final state. First, the most likely final state is extracted from the V values. Since ψ contains the entire path leading up to a state at time t , we simply have to get the value of ψ at time T for the most likely final state.

Expectation-Maximization Algorithm

The Expectation-Maximization algorithm is concerned with constructing an HMM λ given only a sequence of observations $O_1, \dots, O_T$, the hidden state space S , and the emission space E . It is essentially an unsupervised training algorithm. The algorithm consists of two main steps: The Expectation "E" step, and the Maximization "M" step.

Expectation Step

In the expectation step, four values are calculated.

1. $\alpha_t(i)$ The "forward" probabilities, i.e. the probability of seeing state i at time t given $O_1, \dots, O_t$.
2. $\beta_t(i)$ The "backward" probabilities, i.e. the probability of seeing the observations $O_{t+1}, \dots, O_T$ given that we see state i at time t .
3. $\gamma_t(i)$ The "posterior" probabilities, i.e. the probability of seeing state i at time t given the whole observation sequence.
4. $\xi_t(i, j)$ The "joint posterior" probabilities, i.e. the probability of transitioning from state i to state j from time t to time $t + 1$ given the whole observation sequence.

These values are calculated as such:

$$\begin{aligned}\alpha_1(i) &= \pi_i b_i(O_1) \\ \alpha_{t+1}(j) &= \sum_i \alpha_t(i) p_i(j) b_j(O_{t+1}) \\ \beta_T(i) &= 1 \\ \beta_t(j) &= \sum_i \beta_{t+1}(i) p_i(j) b_j(O_{t+1}) \\ \gamma_t(i) &= \frac{\alpha_t(i) \beta_t(i)}{\sum_k \alpha_t(k) \beta_t(k)} \\ \xi_t(i, j) &= \frac{\eta [\alpha_t(i) \cdot p_i(j) \cdot b_j(O_{t+1}) \cdot \beta_{t+1}(j)]}{\sum_i \alpha_t(i)} \\ \eta &= \frac{1}{\sum_i \alpha_T(i)}\end{aligned}$$

Maximization Step

The Maximization step updates the HMM λ by modifying the parameters π, a, b . The following pseudocode illustrates how the values calculated in the E step can be used to update these parameters:

```
 $\pi, A, B \leftarrow$  initialization, transition, emission matrices
obsList  $\leftarrow [O_1, \dots, O_T]$ 

 $\alpha \leftarrow$  forward(obsList,  $\pi, A, B$ )
 $\beta \leftarrow$  backward(obsList,  $\pi, A, B$ )
 $\gamma \leftarrow$  gamma( $\alpha, \beta$ )
 $\xi \leftarrow$  xi(obsList,  $\pi, A, B, \alpha, \beta$ )

 $\pi \leftarrow \gamma_1$ 

A_num  $\leftarrow []$ 
A_denom  $\leftarrow []$ 
for  $j \in S$  do
```

```
    for  $i \in S$  do
        A_num[i][j]  $\leftarrow \sum_{t=1}^{T-1} \xi_t(i, j)$ 
    end for
    A_denom[i]  $\leftarrow \sum_{t=1}^{T-1} \gamma_t(i)$ 
end for
A  $\leftarrow$  A_num / A_denom

B_num  $\leftarrow []$ 
B_denom  $\leftarrow []$ 
for  $i \in S$  do
    for  $k \in E$  do
        B_num[i][k]  $\leftarrow \sum_{t=1, O_t=k}^T \gamma_t(i)$ 
    end for
    B_denom[i]  $\leftarrow \sum_{t=1}^T \gamma_t(i)$ 
end for
B  $\leftarrow$  B_num / B_denom
```

The full implementation of both algorithms can be found here.

Experiments

All experiments used the Brown dataset found in this repository. The experiments in this section tested the following:

1. Amount of training data for supervised training
2. Amount of labeled training data for semi-supervised training
3. Amount of unlabeled training data for semi-supervised training
4. Batch size in semi-supervised training

All experiments were run using the Jupyter Notebook files in the repository. Experiment (1.) tests the performance of just the Viterbi algorithm on an HMM trained using the NLTK library. Note that accuracy represents the per-tag accuracy, i.e. how many tags were assigned correctly. The precision, recall, and F1 scores are averages of the precision, recall, and F1 scores per tag, weighted by how many of each tag was present in the test data.

# of sentences	Accuracy	Precision	Recall	F1
1k	81.1%	82.2%	81.2%	80.3%
10k	90.0%	90.3%	90.1%	89.9%
45k+ (full)	93.6%	93.7%	93.6%	93.6%

Table 1: Performance of Viterbi Algorithm with labeled data

The results show that even on a small amount of labeled data (1000 sentences), the Viterbi algorithm works quite well. In fact, the difference between 10,000 sentences and 45,000 sentences is not very significant, indicating that the increase in accuracy slows down as more data is added. The confusion matrices for each experiment in Figure 1 also support this conclusion.

Experiment (2.) tested how the amount or proportion of labeled/unlabeled data affected results. The findings are summarized in Table 2.

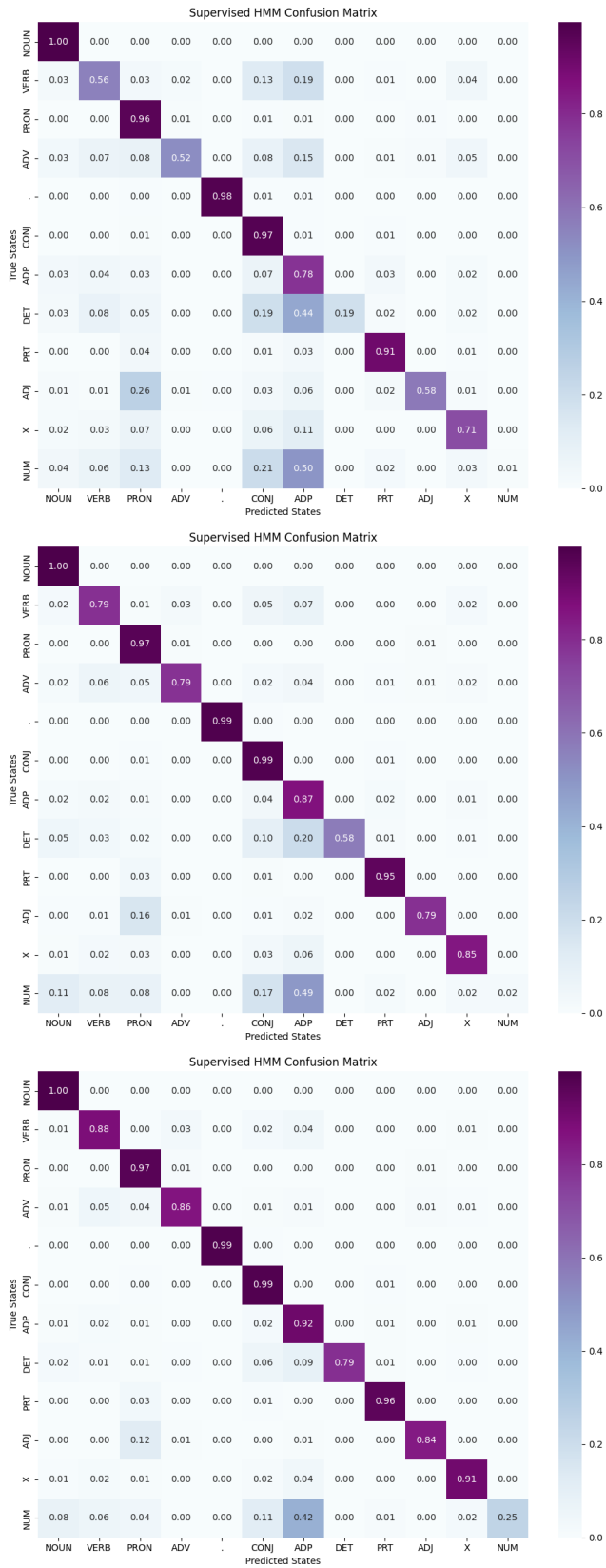


Figure 1: Confusion Matrices for 1k sentences, 10k sentences, and 45k+ sentences

# of labeled sentences	# of unlabeled sentences	Accuracy	Precision	Recall	F1
1k	1k	14.0%	10.0%	14.0%	9.8%
1k	10k	8.4%	10.4%	13.7%	11.5%
10k	1k	15.7%	13.4%	15.8%	6.3%
10k	10k	22.8%	8.4%	22.8%	10.6%

Table 2: Performance of EM and Viterbi based on amount of labeled/unlabeled data

As we can see, the EM algorithm did not perform well. This is expected of course, since it is an unsupervised training algorithm. However, it is interesting to note that it performed the best when the labeled and unlabeled sentences were equal amounts. This is likely due to EM finding different local optima based on the proportion of data. The initial hope with this implementation was that using EM could fine-tune a supervised HMM for unlabeled data. Clearly, with its current state EM worsens the performance of an already supervised HMM. Possible next steps to improve this implementation are discussed in Discussion.

Experiment (3.) tested if training EM using different batch sizes affected the performance. The first experiment was run on 1000 labeled and 1000 unlabeled sentences. The batches are of size 100 and 500 respectively. Figure 2 shows the negative logarithmic probability of all of the batches according to the EM algorithm. We expect that this value is monotonically decreasing with each iteration (Yang, Balakrishnan, and Wainwright 2015). This is generally the case, except for a slight uptick in the first few iterations in the first batch. This is due to the model starting on data from a supervised model, so it is not surprising that the log probability gets worse at first. The models had similar accuracy, 13.7% and 14.0% respectively.

It is promising that both algorithms have a sharp drop in the negative log probability in the early iterations. However, the algorithm does seem to converge quite early, indicating that they find a local minimum. This is likely a limitation in the complexity of the HMM itself, which is discussed in Discussion.

I also ran 10,000 labeled and 10,000 unlabeled sentences using a batch size of 100 to test the convergence point. The result is shown in Figure 3. It is odd to see that the negative log probability dipped below 50. The fact that this only happened in a few iterations and the negative log probability did not continue to decrease monotonically indicates a numerical imprecision issue. This is also discussed in Discussion.

Discussion

This section will discuss improvements to this project to address the issues of numerical imprecision and performance. Due to time constraints, it was not possible to implement these improvements in time, which is why they are not part of Project Description. However, the in-progress code can be viewed on the branches of this repository.

All of the calculations in EM are done in log-space to pre-

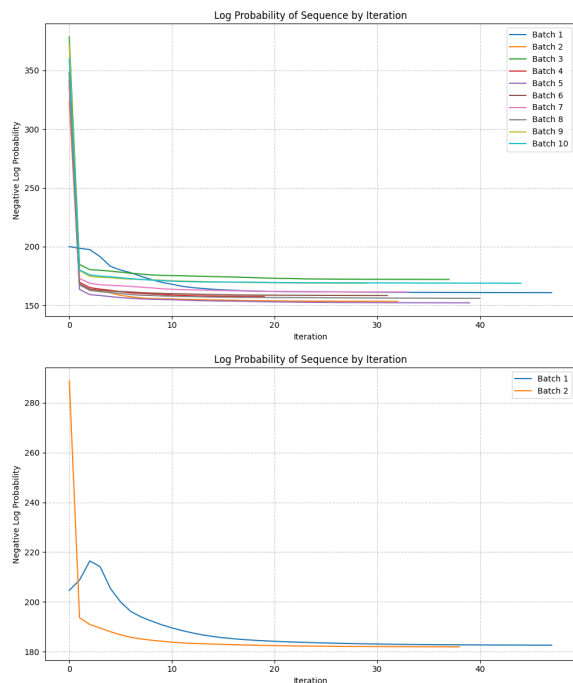


Figure 2: Negative log probability by iteration for batch sizes of 100, 500 on 1000 sentences

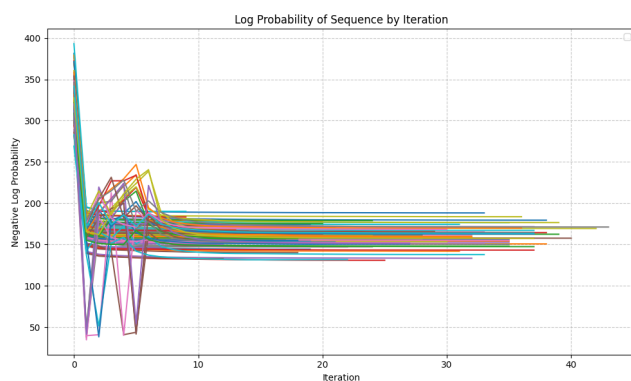


Figure 3: Negative log probability by iteration for batch size 100 on 10,000 sentences

vent underflow. However, Johnson mentions that using real probabilities with rescaling resulted in speed improvements (Johnson). Since his approach uses rescaling explicitly, this would likely get rid of the numerical instability issues.

The main point Johnson mentions in his paper is that while EM does not perform well with 1-to-1 accuracy, it performs significantly better with many-to-1 accuracy (Johnson). This is the main improvement to this project that is currently in progress. This involves creating an HMM with more states than there are tags to be assigned. For example, in this project there are 12 tags. We could initialize an HMM with 36 states (3 per tag) and run EM on it. Since the HMM is more complex, it may pick up on subtleties in the data the original 12 tags do not accurately capture. For example, state 1 may represent past tense verbs while state 2 represents present tense verbs, which both get assigned as “VERB”. After training with these states, we need to run the Viterbi algorithm to map the internal states to the actual tag set. Notice this *does* require using labeled data.

Conclusion

This project has provided valuable insights into the comparative performance of supervised and unsupervised approaches to HMM-based PoS tagging. My experiments with the Brown corpus clearly demonstrated that supervised HMM training consistently outperforms the EM-based unsupervised approach across all evaluation metrics (accuracy, precision, recall, and F1). The significant performance gap between these methods highlights the challenges inherent in unsupervised learning for structured prediction tasks like PoS tagging.

Through my investigation of batch size variations in EM training, I observed that smaller batches led to convergence at smaller log probabilities, suggesting potential limitations in optimization dynamics. This finding points to the sensitivity of the EM algorithm to implementation details and hyperparameter choices.

By completing this project, I gained practical experience in implementing Hidden Markov Models. I learned about the Viterbi and Expectation Maximization algorithms and got to appreciate their structures by implementing them. Even though the results with EM were quite disappointing, these challenges likely stem from the complexity of the tag space, word ambiguity, and the algorithm’s tendency to converge to local optima. This is an opportunity for me to continue exploring the intricacies of the algorithm and improve the project in the future. Moreover, I learned how to run experiments to quantify the performance of AI models, how to calculate relevant metrics and evaluate them.

References

- [Chiche and Yitagesu 2022] Chiche, A., and Yitagesu, B. 2022. Part of speech tagging: a systematic review of deep learning and machine learning approaches. *Journal of Big Data* 9(1):10.
- [Johnson] Johnson, M. Why Doesn’t EM Find Good HMM POS-Taggers?

[Stratos] Stratos, K. Unsupervised Learning 101: the EM for the HMM.

[Yang, Balakrishnan, and Wainwright 2015] Yang, F.; Balakrishnan, S.; and Wainwright, M. J. 2015. Statistical and computational guarantees for the Baum-Welch algorithm. In *2015 53rd Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, 658–665. Monticello, IL: IEEE.